

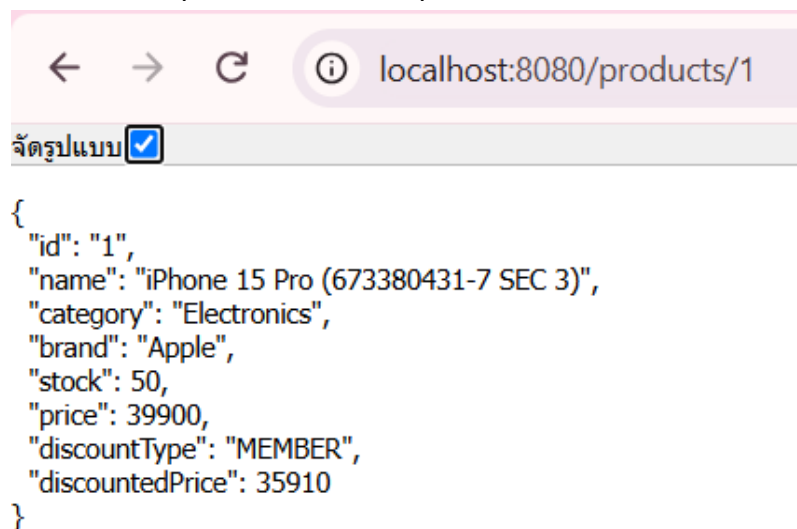
Lab10_Webclient

Screenshot ทดสอบ endpoint

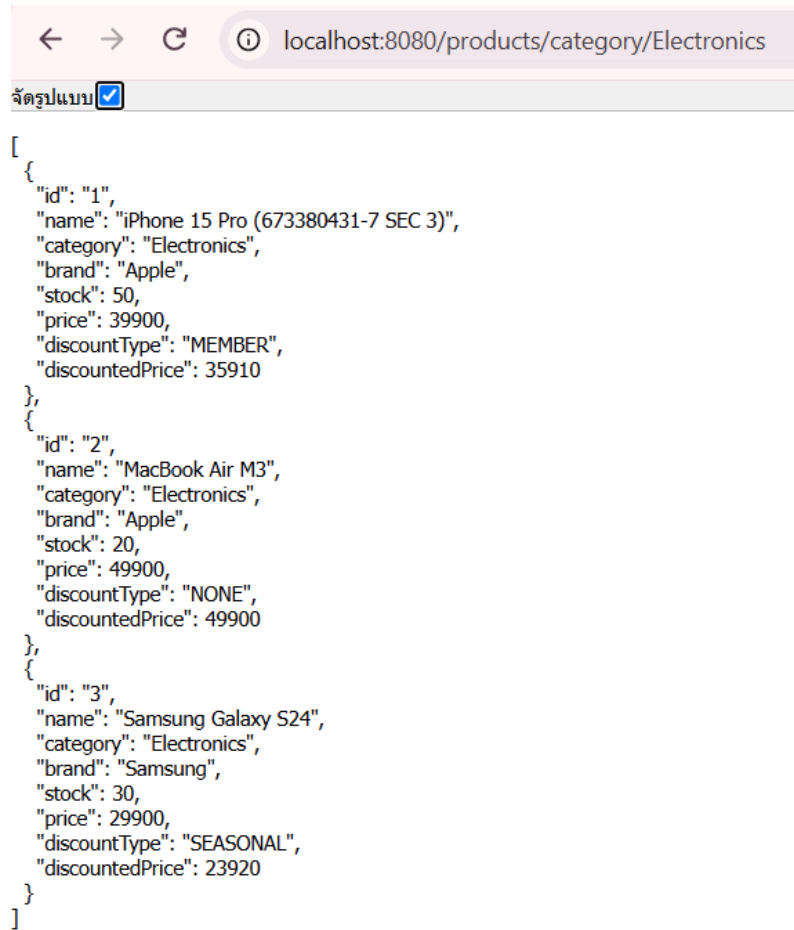
1. ทดสอบ : <http://localhost:8080/products>



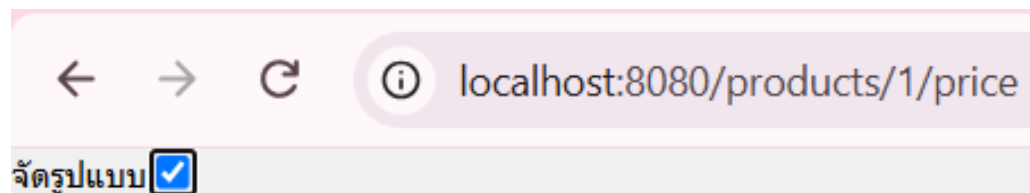
2. ทดสอบ : <http://localhost:8080/products/1>



3.ทดสอบ : <http://localhost:8080/products/category/Electronics>



4.ทดสอบ : <http://localhost:8080/products/1/price>



5.ทดสอบเพิ่มข้อมูล : http://localhost:8080/products

```

<  →  ↻  ⓘ localhost:8080/products
จัดรูปแบบ ☒
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380431-7 SEC 3)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "24bec264-2565-40dd-b952-6778336fdb4d",
    "name": "Sony PlayStation 5",
    "category": "Gaming",
    "brand": "Sony",
    "stock": 15,
    "price": 18900,
    "discountType": "NONE",
    "discountedPrice": 18900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  },
  {
    "id": "6",
    "name": "Sony PlayStation 5",
    "category": "Gaming",
    "brand": "Sony",
    "stock": 15,
    "price": 18900,
    "discountType": "NONE",
    "discountedPrice": 18900
  }
]

```

6.ทดสอบลบข้อมูล : http://localhost:8080/products/2

```

<  →  ↻  ⓘ localhost:8080/products
จัดรูปแบบ ☒
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380431-7 SEC 3)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  },
  {
    "id": "6",
    "name": "Sony PlayStation 5",
    "category": "Gaming",
    "brand": "Sony",
    "stock": 15,
    "price": 18900,
    "discountType": "NONE",
    "discountedPrice": 18900
  }
]

```

ตอบคำถาม

1. Reactive Programming vs Blocking — ต่างกันอย่างไร

ตอบ

Blocking เป็นการทำงานแบบรอคิว Thread ต้องหยุดรอผลลัพธ์จนกว่าจะเสร็จ ทำให้เปลืองทรัพยากรและระบบอาจล่มได้เมื่อคนใช้งานเยอะ

Reactiveเป็นการทำงานแบบNon-blocking สิ่งงานแล้วสลับไปทำอย่างอื่นต่อได้ทันที ใช้ Thread น้อย แต่รองรับผู้ใช้งานพร้อมกันได้เยอะ

2. Mono vs Flux — ใช้กรณีไหน และทำไม

ตอบ Mono<T> ตัวส่งข้อมูล (Publisher) ที่คืนค่าผลลัพธ์ 0 หรือ 1 ค่า ใช้สำหรับสถานการณ์ที่คาดหวังผลลัพธ์เดียว Flux<T> ตัวส่งข้อมูล (Publisher) ที่คืนค่าผลลัพธ์ 0 ถึง N ค่า ใช้สำหรับดึงข้อมูลเป็นกลุ่มหรือลิสต์

3. WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

ตอบ

- .get(), .post(), .delete(): ตั้งค่า HTTP Method ว่าต้องการทำคำสั่งประเภทใด
- .uri(...): ระบุปลายทาง URL หรือ Path ของ API ที่ต้องการไปดึงหรือส่งข้อมูล
- .retrieve(): สั่งให้ระบบทำการยิง Request ไปยังเซิร์ฟเวอร์เพื่อดึงข้อมูลกลับมา
- .bodyToMono(...) / .bodyToFlux(...): แปลงข้อมูลผลลัพธ์ที่ได้จาก Response (มักเป็น JSON) กลับมาเป็น Java Object 1 ชิ้น (Mono) หรือหลายชิ้น (Flux)

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

ตอบ

ProductRepository.java

```
Java
public class ProductRepository {
    private final Map<String, Product> store = new ConcurrentHashMap<>();

    // ... Constructor สำหรับจำลองข้อมูลเริ่มต้น ...

    public Mono<Product> findById(String id) {
        Product product = store.get(id); // ดึงข้อมูลจาก Map ด้วย ID
        if (product != null) {
            return Mono.just(product); // ถ้าเจอ ให้ห่อ object ด้วย Mono.just() เพื่อคืน 1
            ค่า
        }
    }
}
```

```

    }
    return Mono.empty(); // ถ้าไม่เจอ คืนค่าความว่างเปล่า
}

public Flux<Product> findAll() {
    return Flux.fromIterable(store.values()); // แปลง Collection (ข้อมูลทั้งหมดใน
Map) ให้เป็น Flux
}

public Mono<Product> save(Product product) {
    store.put(product.getId(), product); // บันทึกข้อมูลลง Map
    return Mono.just(product); // คืนค่า Product ที่เพิ่งบันทึกไป
}

public Mono<Void> deleteById(String id) {
    store.remove(id); // ลบข้อมูลออกจาก Map
    return Mono.empty(); // ไม่มีการคืนค่าข้อมูลกลับไป จึงใช้ Mono.empty() หรือ Mono<Void>
}

public Flux<Product> findByCategory(String category) {
    return findAll() // ดึงข้อมูลทั้งหมดมาก่อนเป็น Flux
        .filter(p -> p.getCategory().equalsIgnoreCase(category)); // กรอง
เอาเฉพาะชิ้นที่ category ตรงกับที่ส่งมา
}
}

```

ProductService.java

```

Java
@Service
public class ProductService {
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository; // Inject Repository เข้ามาใช้งาน
    }

    public Mono<Product> getById(String id) {
        return repository.findById(id) // เรียกหา ID
            .switchIfEmpty(Mono.error(new RuntimeException("Product not found:
" + id))); // ถ้าข้างหน้าได้ Mono.empty() ให้พ่น Error ออกมา
    }
}

```

```

public Flux<Product> getAll() {
    return repository.findAll(); // ส่งต่อข้อมูลทั้งหมดจาก Repository
}

public Mono<Product> save(Product product) {
    if (product.getId() == null || product.getId().isEmpty()) {
        product.setId(UUID.randomUUID().toString()); // ถ้าไม่มี ID ส่งมา ให้สุ่มสร้าง
        ID แบบ UUID ใหม่
    }
    return repository.save(product); // ส่งไปบันทึก
}

public Mono<Void> delete(String id) {
    return repository.deleteById(id); // ส่งคำสั่งลบไปยัง Repository
}

public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category); // ส่งต่อผลลัพธ์การกรองหมวดหมู่
}

public Mono<Double> getDiscountedPrice(String id) {
    return getById(id) // ค้นหา Product ตาม ID
        .map(Product::getDiscountedPrice); // แปลงค่า (Transform) จาก Object
        Product ให้เหลือแค่ค่า Double (ราคา)
}
}

```

ProductController.java

```

Java
@RestController
@RequestMapping("/products") // ตั้งค่า Path หลักเป็น /products
public class ProductController {
    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service; // Inject Service เข้ามาใช้งาน
    }

    @GetMapping("/{id}") // รับ HTTP GET แบบแนบ Path Variable
    public Mono<Product> getById(@PathVariable String id) {
        return service.getById(id); // คืนข้อมูล 1 ชิ้น
    }
}

```

```

@GetMapping // รับ HTTP GET เข้า /products ปกติ
public Flux<Product> getAll() {
    return service.getAll(); // คืนข้อมูลเป็นลิสต์ทั้งหมด
}

@PostMapping // รับ HTTP POST สำหรับการสร้างข้อมูลใหม่
public Mono<Product> save(@RequestBody Product product) {
    return service.save(product); // ดึงข้อมูลจาก Request Body ไปบันทึก
}

@DeleteMapping("/{id}") // รับ HTTP DELETE สำหรับลบข้อมูล
public Mono<Void> delete(@PathVariable String id) {
    return service.delete(id);
}

@GetMapping("/category/{category}") // รับ HTTP GET พร้อมแนบหมวดหมู่
public Flux<Product> getByCategory(@PathVariable String category) {
    return service.getByCategory(category);
}

@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    return service.getDiscountedPrice(id); // คืนเฉพาะราคาที่ลดแล้ว
}
}

```

ProductWebClient.java

```

Java
@Component
public class ProductWebClient {
    private final WebClient webClient;

    public ProductWebClient(WebClient.Builder webClientBuilder) {
        // สร้าง WebClient และชี้เป้าหมายไปที่ URL หลักของ API
        this.webClient =
webClientBuilder.baseUrl("http://localhost:8080").build();
    }

    public Mono<Product> getProductById(String id) {
        return webClient.get() // เรียกใช้ HTTP GET
            .uri("/products/{id}", id) // ใส่ path และยัดค่าตัวแปร id ลงไป
    }
}

```

```

        .retrieve() // สืบดึงข้อมูล
        .bodyToMono(Product.class); // แปลงผลลัพธ์ (JSON) กลับมาเป็น Class
Product 1 ขึ้น
    }

    public Flux<Product> getAllProducts() {
        return webClient.get()
            .uri("/products")
            .retrieve()
            .bodyToFlux(Product.class); // แปลงผลลัพธ์กลับมาเป็น Class Product หลาย
ขึ้น (Flux)
    }

    public Mono<Product> createProduct(Product product) {
        return webClient.post() // เรียกใช้ HTTP POST
            .uri("/products")
            .bodyValue(product) // แนบ Object ลงไปใน Request Body
            .retrieve()
            .bodyToMono(Product.class);
    }

    public Mono<Void> deleteProduct(String id) {
        return webClient.delete() // เรียกใช้ HTTP DELETE
            .uri("/products/{id}", id)
            .retrieve()
            .bodyToMono(Void.class); // รับคืนเป็น Void เพราะไม่มีข้อมูลคืนมา
    }

    public Flux<Product> getProductsByCategory(String category) {
        return webClient.get()
            .uri("/products/category/{category}", category)
            .retrieve()
            .bodyToFlux(Product.class);
    }

    public Mono<Double> getDiscountedPrice(String id) {
        return webClient.get()
            .uri("/products/{id}/price", id)
            .retrieve()
            .bodyToMono(Double.class); // แปลงผลลัพธ์เป็นค่า Double เดียวๆ
    }
}

```